

Testing

How To Write Automated Tests

What is testing?

“Clicking through the app to make sure nothing’s broken.”

“Writing code that executes individual pieces of your app to make sure they’re not broken.”

Why write automated tests?

- Reliability
- Documentation
- Refactorability
- Confirm that ChatGPT isn't breaking everything
- Because in six months, you will wish you had

Types of Tests

- Unit Tests
 - Is this Lego piece broken?
- Integration Tests
 - Do these two Lego pieces fit together?
- End to End (e2e) Tests
 - Is my Lego tower still standing?

Testing Libraries & Tools



Anatomy of a Test

```
it("should be able to add two", () => {  
  expect(BasicArithmetic.addTwo(2)).toBe(4);  
});
```

it takes a label (string) and a test callback function

expect throws an error if the expectations are not met

If no errors are thrown by the end of the **it** block, the test passes

Anatomy of a Test

```
describe("Basic Arithmetic", () => {  
  it("should be able to add two", () => {  
    expect(BasicArithmetic.addTwo(2)).toBe(4);  
  });  
  
  it("should be able to subtract two", () => {  
    expect(BasicArithmetic.subtractTwo(2)).toBe(0);  
  });  
});
```

describe also takes a label and a callback function

describe is used to organize individual tests into groups

describe blocks can be nested within others to create a well organized testing structure

Anatomy of a Test

```
describe("Basic Arithmetic", () => {  
  beforeEach(() => {  
    BasicArithmetic.resetAllNumbers;  
  });  
  
  it("should be able to add two", () => {  
    expect(BasicArithmetic.addTwo(2)).toBe(4);  
  });  
  
  it("should be able to subtract two", () => {  
    expect(BasicArithmetic.subtractTwo(2)).toBe(0);  
  });  
});
```

beforeEach also allows us to run code before each **it** block runs

There's also **afterEach**, **beforeAll**, and **afterAll**

It's important to make sure that each test can run in isolation from all the others

Mocking

```
describe("RestaurantList", () => {
  const mockFetchAllRestaurants = jest.fn();

  beforeEach(() => {
    mockFetchAllRestaurants.mockClear();
  });

  it('renders "No restaurants found" when restaurants array is empty', () => {
    render(
      <RestaurantList
        restaurants={[]}
        fetchAllRestaurants={mockFetchAllRestaurants}
      />
    );

    expect(screen.getByText("No restaurants found")).toBeInTheDocument();
    expect(mockFetchAllRestaurants).toHaveBeenCalledTimes(1);
  });
});
```

It is often necessary to simulate different scenarios, called “mocking”

React components can be easily tested by passing in different props

We can also force mock functions to return specific data (or even throw errors)

Testing Practice

